

PAYMENT TRACK, DAY 3: invoices, BACKEND

E-Learning Platform (Spring Boot 3.2.5, Java 17). **5 new files, 2 files replaced, 5 lines in `application.properties`, 1 line in `SecurityConfig`**. One new table, `invoices`. **No new Maven dependency**: the PDF file is written by 150 lines of our own code (explained below), so there is no library to download.

Prerequisites: Day 1 and Day 2 work.

By the end of Day 3: the moment a payment is verified, the customer gets an **invoice** with a number like `INV-2026-000001`. He can see it on screen and download it as a **PDF file** from the browser. Nobody else can read it, except the admin.

0. What you get, and the rules I chose

Rule	Why
One invoice per verified payment , made automatically right after the payment is confirmed	the payment is the thing that was paid for
The invoice is a snapshot : course names, prices, the buyer's name and e-mail are copied into it	later edits to a course must never change an invoice already issued
Invoice number = <code>INV</code> + year + the database id, six digits (<code>INV-2026-000001</code>)	unique, easy to read, no extra counter to keep in step
The lines are the amount actually paid , split over the courses, and always add up to the total	uses <code>Money.allocate</code> from Day 2
GST is a setting, off by default (<code>invoice.gst-percent=0</code>). When you set it (for example 18), the invoice says "Tax invoice" and shows how much of the price is tax	the course price already includes tax; we only show the split, we never add tax on top
A customer can read only his own invoices; a stranger's id answers "not found" (so ids cannot be guessed); an admin can read all	privacy
If the invoice fails to be made, the student is still enrolled ; the invoice is made the next time it is asked for	bookkeeping must never take away what was paid for
Invoices exist for Razorpay payments only	an order saved with the old no-payment button has no payment to bill. With <code>razorpay.enforce=true</code> every order is a paid one

Not tax advice. I do not know your company's GST status. Whether you must charge GST, at what rate, and what must be printed (your GSTIN, address, HSN/SAC code) is for your company's finance or CA to decide. The guide gives you the settings; the numbers on the invoice are only as right as what you put in them.

1. The ideas, from zero (tied to your code)

Snapshot. A bank statement does not change if the shop later renames a product. Same here:

`Invoice.lines` stores one line per course as text, `courseId|title|paise`, one per line.

`InvoiceService.lines()` reads it back. We did not make a separate lines table because an invoice is never edited, only read.

Tax inside the price. If a course costs Rs 500 and GST is 18% **included**, the tax part is $500 \times 18 / 118$ = Rs 76.27 and the part before tax is Rs 423.73. `Money.taxInside(total, percent)` computes that in whole paise, rounded half up; `taxable = total - tax`, so `taxable + tax` is always exactly the total paid.

Invoice number. The database gives every new row an id only when it is saved. So `ensureForPayment` saves the invoice with a temporary unique number, reads the id, then saves the real number (INV-2026- + id padded to six digits). The year comes from the payment date in Indian time.

Idempotent again. `payment_id` in `invoices` is **unique**. Calling `ensureForPayment` twice for the same payment returns the same invoice; two requests at the same moment cannot make two (the loser catches the duplicate-key error and uses the winner's invoice).

What a PDF file really is. It looks like magic, but a simple PDF is just **text** plus a **table of byte positions**:

```
%PDF-1.4
1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj      <- object 1: the book
2 0 obj << /Type /Pages /Kids [5 0 R] /Count 1 >> endobj  <- object 2: the list of pages
3 0 obj << ... /BaseFont /Helvetica >> endobj           <- a built-in font
5 0 obj << /Type /Page ... /Contents 6 0 R >> endobj     <- one page
6 0 obj << /Length 412 >> stream
  BT /F2 20 Tf 50 790 Td (INVOICE) Tj ET                <- "draw this text at x=50, y=790"
  0.6 w 50 636 m 545 636 l S                             <- "draw a line"
endstream endobj
xref <- for each object: how many bytes from the start of the file it begins
startxref 1234 <- where the xref table begins
%%EOF
```

`InvoicePdf.assemble` writes exactly that, counting the bytes of each object so the `xref` table is right. Coordinates start at the **bottom-left** corner of the page (A4 is 595 x 842 points), which is why the title is at y=790. We use the built-in **Helvetica** font, which every PDF viewer has, so we embed nothing. The price of that simplicity: only plain characters print (an accented letter becomes ?), and the amounts are written `Rs 500.00` because the rupee sign is not in that font. Names longer than 70 characters are cut with `...`. More than 24 courses start a second page.

Downloading with a login token. The PDF endpoint needs the login token like every other endpoint. A plain link cannot send the token, so the Angular app asks for the file as a **blob** with the token and saves it (see the frontend guide). The server answers with `Content-Type: application/pdf` and `Content-Disposition: attachment; filename="invoice-INV-2026-000001.pdf"`.

2. What goes where

File	What it is
<code>model/Invoice.java</code>	the invoice snapshot (new table <code>invoices</code>)
<code>repository/InvoiceRepo.java</code>	data access
<code>repository/PaymentRepo.java</code>	replaced : one new method, <code>findByOrderId</code>
<code>pay/InvoicePdf.java</code>	writes the PDF bytes

File	What it is
service/InvoiceService.java	make, list, read, PDF
controller/InvoiceController.java	GET /api/invoice/my, /by-order/{orderId}, /{id}, /{id}/pdf
service/PaymentService.java	replaced (final version: calls both the invoice and the earnings)

2a. application.properties : add at the very end

```
# Day 3: invoices. The seller block is printed on every invoice.
invoice.prefix=INV
invoice.seller-name=${INVOICE_SELLER_NAME:Online E-Learning Platform}
# write a new line as \n (for example: 12 Main Road\nNellore, Andhra Pradesh 524001)
invoice.seller-address=${INVOICE_SELLER_ADDRESS:}
# leave both empty / 0 if the company does not charge GST
invoice.seller-gstin=${INVOICE_SELLER_GSTIN:}
invoice.gst-percent=${INVOICE_GST_PERCENT:0}
```

2b. SecurityConfig.java : one line

In the final SecurityConfig of the master plan (section 8) it is already there. If you maintain your own file, add this line **above** `.anyRequest().authenticated()`:

```
.requestMatchers("/api/invoice/**").hasAnyAuthority("CUSTOMER", "ADMIN")
```

Without it an instructor could reach the invoice endpoints and get a confusing error; with it he gets a clean 403.

3. Complete code

springapp/src/main/java/com/examly/springapp/model/Invoice.java (new file)

```
package com.examly.springapp.model;

import jakarta.persistence.*;
import java.time.Instant;

/**
 * The invoice (tax receipt) of one verified payment. It is a SNAPSHOT: the course names, prices and the
 * buyer's
 * name are copied in when the invoice is made, so later edits to a course never change an invoice already
 * issued.
 * New table "invoices". One payment has at most one invoice.
 */
@Entity
@Table(name = "invoices")
public class Invoice {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 40)
    private String invoiceNumber;
    @Column(nullable = false, unique = true)
    private Long paymentId;
    private Long orderId;
    private Long customerId;
    @Column(length = 120)
```

```

private String customerName;
@Column(length = 120)
private String customerEmail;
@Column(length = 80)
private String razorpayPaymentId;

/** One line per course, formatted "courseId|title|paise" and joined with a new line. */
@Column(length = 4000)
private String lines;

/** Amount before tax, tax, and the total the customer paid (all in paise). total = taxable + tax. */
private long taxablePaise;
private long taxPaise;
private long totalPaise;
/** GST rate in percent that was used (0 means no tax lines). */
private int gstPercent;
private Instant issuedAt;

public Invoice() {}

public Long getId() { return id; }
public String getInvoiceNumber() { return invoiceNumber; }
public void setInvoiceNumber(String v) { invoiceNumber = v; }
public Long getPaymentId() { return paymentId; }
public void setPaymentId(Long v) { paymentId = v; }
public Long getOrderId() { return orderId; }
public void setOrderId(Long v) { orderId = v; }
public Long getCustomerId() { return customerId; }
public void setCustomerId(Long v) { customerId = v; }
public String getCustomerName() { return customerName; }
public void setCustomerName(String v) { customerName = v; }
public String getCustomerEmail() { return customerEmail; }
public void setCustomerEmail(String v) { customerEmail = v; }
public String getRazorpayPaymentId() { return razorpayPaymentId; }
public void setRazorpayPaymentId(String v) { razorpayPaymentId = v; }
public String getLines() { return lines; }
public void setLines(String v) { lines = v; }
public long getTaxablePaise() { return taxablePaise; }
public void setTaxablePaise(long v) { taxablePaise = v; }
public long getTaxPaise() { return taxPaise; }
public void setTaxPaise(long v) { taxPaise = v; }
public long getTotalPaise() { return totalPaise; }
public void setTotalPaise(long v) { totalPaise = v; }
public int getGstPercent() { return gstPercent; }
public void setGstPercent(int v) { gstPercent = v; }
public Instant getIssuedAt() { return issuedAt; }
public void setIssuedAt(Instant v) { issuedAt = v; }
}

```

springapp/src/main/java/com/examly/springapp/repository/InvoiceRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.Invoice;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for invoices. */
@Repository
public interface InvoiceRepo extends JpaRepository<Invoice, Long> {
    Optional<Invoice> findByPaymentId(Long paymentId);
    List<Invoice> findByCustomerIdOrderByIdDesc(Long customerId);
}

```

springapp/src/main/java/com/examly/springapp/repository/PaymentRepo.java (REPLACE the Day 1 file: one new method)

```
package com.examly.springapp.repository;

import com.examly.springapp.model.Payment;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for Razorpay payment attempts. */
@Repository
public interface PaymentRepo extends JpaRepository<Payment, Long> {
    Optional<Payment> findByRazorpayOrderId(String razorpayOrderId);
    Optional<Payment> findByOrderId(Long orderId);
}
```

springapp/src/main/java/com/examly/springapp/pay/InvoicePdf.java (new file)

```
package com.examly.springapp.pay;

import java.io.ByteArrayOutputStream;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

/**
 * Draws an invoice as a real PDF file, with no library: a PDF is plain text with a table of byte positions
 * at
 * the end, and for a one-page-per-24-lines invoice that is a few dozen lines of code.
 * Uses the built-in Helvetica font, so only plain Latin characters are printed (other characters become
 * '?',
 * and the rupee amount is written as "Rs").
 */
public final class InvoicePdf {
    private InvoicePdf() {}

    /** Everything printed on the invoice. */
    public static final class Data {
        public String sellerName = "", sellerAddress = "", sellerGstin = "";
        public String number = "", date = "", paymentRef = "";
        public String customerName = "", customerEmail = "";
        /** Each line: {title, amount in paise as text}. */
        public List<String[]> lines = new ArrayList<>();
        public long taxable, tax, total;
        public int gstPercent;
    }

    private static final int LINES_PER_PAGE = 24;

    public static byte[] render(Data d) {
        List<String> pages = new ArrayList<>();
        int n = d.lines.size();
        int pageCount = Math.max(1, (n + LINES_PER_PAGE - 1) / LINES_PER_PAGE);
        for (int p = 0; p < pageCount; p++) {
            pages.add(pageContent(d, p, pageCount));
        }
        return assemble(pages);
    }

    // ----- one page of drawing commands

    private static String pageContent(Data d, int page, int pageCount) {
        StringBuilder c = new StringBuilder();
        text(c, 50, 790, 20, "F2", d.gstPercent > 0 ? "TAX INVOICE" : "INVOICE");
        text(c, 50, 768, 11, "F2", d.sellerName);
        int y = 754;
    }
}
```

```

for (String part : d.sellerAddress.split("\\n")) {
    if (!part.isBlank()) { text(c, 50, y, 9, "F1", part.trim()); y -= 12; }
}
if (!d.sellerGstin.isBlank()) { text(c, 50, y, 9, "F1", "GSTIN: " + d.sellerGstin); }

text(c, 360, 790, 10, "F2", "Invoice no.");
text(c, 440, 790, 10, "F1", d.number);
text(c, 360, 774, 10, "F2", "Date");
text(c, 440, 774, 10, "F1", d.date);
text(c, 360, 758, 10, "F2", "Payment ref.");
text(c, 440, 758, 10, "F1", d.paymentRef);
if (pageCount > 1) text(c, 360, 742, 9, "F1", "Page " + (page + 1) + " of " + pageCount);

text(c, 50, 690, 10, "F2", "Billed to");
text(c, 50, 676, 11, "F1", d.customerName);
text(c, 50, 662, 10, "F1", d.customerEmail);

// table header
line(c, 50, 636, 545, 636);
text(c, 55, 622, 10, "F2", "Course");
text(c, 470, 622, 10, "F2", "Amount (Rs)");
line(c, 50, 614, 545, 614);

int from = page * LINES_PER_PAGE;
int to = Math.min(d.lines.size(), from + LINES_PER_PAGE);
int ty = 598;
for (int i = from; i < to; i++) {
    String[] row = d.lines.get(i);
    text(c, 55, ty, 10, "F1", clip(row[0], 70));
    textRight(c, 540, ty, 10, "F1", row[1]);
    ty -= 16;
}
line(c, 50, ty + 8, 545, ty + 8);

if (page == pageCount - 1) {
    int sy = ty - 10;
    if (d.gstPercent > 0) {
        text(c, 330, sy, 10, "F1", "Amount before tax");
        textRight(c, 540, sy, 10, "F1", Money.rupees(d.taxable));
        sy -= 16;
        text(c, 330, sy, 10, "F1", "GST " + d.gstPercent + "% (included in price)");
        textRight(c, 540, sy, 10, "F1", Money.rupees(d.tax));
        sy -= 16;
    }
    text(c, 330, sy, 12, "F2", "Total paid");
    textRight(c, 540, sy, 12, "F2", "Rs " + Money.rupees(d.total));
    text(c, 50, 60, 9, "F1", "This is a computer generated invoice for a payment made online. No
signature is required.");
} else {
    text(c, 50, 60, 9, "F1", "Continued on the next page");
}
return c.toString();
}

private static String clip(String s, int max) {
    String t = ascii(s);
    return t.length() <= max ? t : t.substring(0, max - 3) + "...";
}

private static void text(StringBuilder c, int x, int y, int size, String font, String s) {
    c.append("BT /").append(font).append(' ').append(size).append(" Tf ")
        .append(x).append(' ').append(y).append(" Td (").append(escape(s)).append(") Tj ET\\n");
}

/** Right aligned: Helvetica digits are about 0.556 em wide, which is accurate enough for numbers. */
private static void textRight(StringBuilder c, int xRight, int y, int size, String font, String s) {
    int w = (int) Math.round(ascii(s).length() * size * 0.556);
    text(c, xRight - w, y, size, font, s);
}

private static void line(StringBuilder c, int x1, int y1, int x2, int y2) {

```

```

        c.append("0.6 w ").append(x1).append(' ').append(y1).append(" m ").append(x2).append('
').append(y2).append(" l S\n");
    }

    static String ascii(String s) {
        if (s == null) return "";
        StringBuilder sb = new StringBuilder();
        for (char ch : s.toCharArray()) {
            sb.append(ch >= 32 && ch < 127 ? ch : '?');
        }
        return sb.toString();
    }

    static String escape(String s) {
        return ascii(s).replace("\\", "\\").replace("(", "\\(").replace(")", "\\)");
    }

    // ----- file structure

    private static byte[] assemble(List<String> contents) {
        List<byte[]> objs = new ArrayList<>(); // object number = index + 1
        int pages = contents.size();
        // 1 catalog, 2 page tree, 3 Helvetica, 4 Helvetica-Bold, then (page, content) pairs
        StringBuilder kids = new StringBuilder();
        for (int i = 0; i < pages; i++) kids.append(5 + i * 2).append(" 0 R ");
        objs.add(b("<< /Type /Catalog /Pages 2 0 R >>"));
        objs.add(b("<< /Type /Pages /Kids [" + kids.toString().trim() + "] /Count " + pages + " >>"));
        objs.add(b("<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica /Encoding /WinAnsiEncoding >>"));
        objs.add(b("<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica-Bold /Encoding /WinAnsiEncoding
>>"));
        for (int i = 0; i < pages; i++) {
            int contentObj = 6 + i * 2;
            objs.add(b("<< /Type /Page /Parent 2 0 R /MediaBox [0 0 595 842] /Resources << /Font << /F1 3 0
R /F2 4 0 R >> >> /Contents " + contentObj + " 0 R >>"));
            byte[] stream = contents.get(i).getBytes(StandardCharsets.ISO_8859_1);
            objs.add(concat(b("<< /Length " + stream.length + " >>\nstream\n"), stream, b("endstream"))));
        }
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        write(out, b("%PDF-1.4\n"));
        int[] offsets = new int[objs.size()];
        for (int i = 0; i < objs.size(); i++) {
            offsets[i] = out.size();
            write(out, b((i + 1) + " 0 obj\n"));
            write(out, objs.get(i));
            write(out, b("\nendobj\n"));
        }
        int xref = out.size();
        StringBuilder x = new StringBuilder("xref\n0 " + (objs.size() + 1) + "\n0000000000 65535 f \n");
        for (int off : offsets) x.append(String.format("%010d 00000 n \n", off));
        x.append("trailer\n<< /Size ").append(objs.size() + 1).append(" /Root 1 0 R
>>\nstartxref\n").append(xref).append("\n%%EOF\n");
        write(out, b(x.toString()));
        return out.toByteArray();
    }

    private static byte[] b(String s) { return s.getBytes(StandardCharsets.ISO_8859_1); }
    private static void write(ByteArrayOutputStream o, byte[] bytes) { o.write(bytes, 0, bytes.length); }
    private static byte[] concat(byte[]... parts) {
        ByteArrayOutputStream o = new ByteArrayOutputStream();
        for (byte[] p : parts) write(o, p);
        return o.toByteArray();
    }
}

```

springapp/src/main/java/com/examly/springapp/service/InvoiceService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.PayDtos.*;

```

```

import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.Customer;
import com.examly.springapp.model.Invoice;
import com.examly.springapp.model.Payment;
import com.examly.springapp.pay.InvoicePdf;
import com.examly.springapp.pay.Money;
import com.examly.springapp.repository.CourseRepo;
import com.examly.springapp.repository.CustomerRepo;
import com.examly.springapp.repository.InvoiceRepo;
import com.examly.springapp.repository.PaymentRepo;
import java.time.Instant;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Optional;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;

/**
 * Makes the invoice of a verified payment and hands it out as data or as a PDF file.
 * A customer can see only his own invoices; an admin can see all of them.
 */
@Service
public class InvoiceService {
    private static final Logger log = LoggerFactory.getLogger(InvoiceService.class);
    private static final ZoneId IST = ZoneId.of("Asia/Kolkata");
    private static final DateTimeFormatter DATE = DateTimeFormatter.ofPattern("dd MMM yyyy",
Locale.ENGLISH);

    private final InvoiceRepo invoices;
    private final PaymentRepo payments;
    private final CustomerRepo customers;
    private final CourseRepo courses;
    private final AccessService access;
    private final String prefix, sellerName, sellerAddress, sellerGstin;
    private final int gstPercent;

    public InvoiceService(InvoiceRepo invoices, PaymentRepo payments, CustomerRepo customers, CourseRepo
courses,
                        AccessService access,
                        @Value("${invoice.prefix:INV}") String prefix,
                        @Value("${invoice.seller-name:Online E-Learning Platform}") String sellerName,
                        @Value("${invoice.seller-address:}") String sellerAddress,
                        @Value("${invoice.seller-gstin:}") String sellerGstin,
                        @Value("${invoice.gst-percent:0}") int gstPercent) {
        this.invoices = invoices;
        this.payments = payments;
        this.customers = customers;
        this.courses = courses;
        this.access = access;
        this.prefix = prefix;
        this.sellerName = sellerName;
        this.sellerAddress = sellerAddress;
        this.sellerGstin = sellerGstin;
        this.gstPercent = Math.max(0, Math.min(40, gstPercent));
    }

    // ----- making the invoice

    /** Makes the invoice for a PAID payment, or returns the one that already exists. Safe to call many
times. */
    public Invoice ensureForPayment(Payment payment) {

```

```

        if (payment == null || !Payment.PAID.equals(payment.getStatus())) return null;
        Optional<Invoice> existing = DatabaseOperationException.guard("loading an invoice", () ->
invoices.findByPaymentId(payment.getPaymentId()));
        if (existing.isPresent()) return existing.get();

        Customer customer = DatabaseOperationException.guard("loading a customer", () ->
customers.findById(payment.getCustomerId()).orElse(null);
        List<Long> ids = new ArrayList<>();
        for (String s : payment.getCourseIds().split(",")) {
            if (!s.isBlank()) ids.add(Long.valueOf(s.trim()));
        }
        List<String> titles = new ArrayList<>();
        List<Long> weights = new ArrayList<>();
        for (Long id : ids) {
            Course c = DatabaseOperationException.guard("loading a course", () ->
courses.findById(id)).orElse(null);
            titles.add(c == null ? "Course " + id : c.getCourseType());
            weights.add(c == null || c.getCoursePrice() == null ? 0L : Math.round(c.getCoursePrice() *
100));
        }
        List<Long> amounts = Money.allocate(payment.getAmountPaise(), weights);
        StringBuilder lines = new StringBuilder();
        for (int i = 0; i < ids.size(); i++) {
            if (i > 0) lines.append('\n');
            lines.append(ids.get(i)).append('|').append(titles.get(i).replace('|', '/').replace('\n', '
')).append('|').append(amounts.get(i));
        }

        Invoice inv = new Invoice();
        inv.setInvoiceNumber("TMP-" + UUID.randomUUID()); // unique placeholder until the database
gives us the id
        inv.setPaymentId(payment.getPaymentId());
        inv.setOrderId(payment.getOrderId());
        inv.setCustomerId(payment.getCustomerId());
        inv.setCustomerName(customer == null ? "Customer" : customer.getCustomerName());
        inv.setCustomerEmail(customer == null || customer.getUser() == null ? "" :
customer.getUser().getEmail());
        inv.setRazorpayPaymentId(payment.getRazorpayPaymentId());
        inv.setLines(lines.toString());
        long total = payment.getAmountPaise();
        long tax = Money.taxInside(total, gstPercent);
        inv.setTotalPaise(total);
        inv.setTaxPaise(tax);
        inv.setTaxablePaise(total - tax);
        inv.setGstPercent(gstPercent);
        Instant now = payment.getPaidAt() == null ? Instant.now() : payment.getPaidAt();
        inv.setIssuedAt(now);
        try {
            Invoice saved = invoices.save(inv);
            saved.setInvoiceNumber(prefix + "-" + now.atZone(IST).getYear() + "-" + String.format("%06d",
saved.getId()));
            saved = invoices.save(saved);
            log.info("Invoice {} created for payment {}", saved.getInvoiceNumber(),
payment.getPaymentId());
            return saved;
        } catch (DataIntegrityViolationException dup) {
            // two requests made it at the same moment: the other one won, use its invoice
            return invoices.findByPaymentId(payment.getPaymentId()).orElseThrow(() -> dup);
        }
    }

    // ----- reading

    public List<InvoiceView> listMine() {
        Customer me = access.currentCustomer();
        List<InvoiceView> out = new ArrayList<>();
        for (Invoice i : DatabaseOperationException.guard("loading invoices", () ->
invoices.findByCustomerIdOrderByDesc(me.getCustomerId())) {
            out.add(view(i));
        }
        return out;
    }

```

```

    }

    public InvoiceView get(Long id) {
        return view(load(id));
    }

    /** The invoice of an order. If the payment was verified but the invoice is missing (it failed once),
    it is made now. */
    public InvoiceView getByOrder(Long orderId) {
        Payment p = DatabaseOperationException.guard("loading a payment", () ->
payments.findByOrderId(orderId))
            .orElseThrow(() -> new ResourceNotFoundException("There is no invoice for this order"));
        Invoice inv = ensureForPayment(p);
        if (inv == null) throw new ResourceNotFoundException("There is no invoice for this order");
        check(inv);
        return view(inv);
    }

    public byte[] pdf(Long id) {
        Invoice inv = load(id);
        InvoicePdf.Data d = new InvoicePdf.Data();
        d.sellerName = sellerName;
        d.sellerAddress = sellerAddress.replace("\\n", "\n");
        d.sellerGstin = sellerGstin;
        d.number = inv.getInvoiceNumber();
        d.date = DATE.format(inv.getIssuedAt().atZone(IST));
        d.paymentRef = inv.getRazorpayPaymentId() == null ? "" : inv.getRazorpayPaymentId();
        d.customerName = inv.getCustomerName();
        d.customerEmail = inv.getCustomerEmail();
        for (InvoiceLineView l : lines(inv)) {
            d.lines.add(new String[]{l.title, Money.rupees(l.paise)});
        }
        d.taxable = inv.getTaxablePaise();
        d.tax = inv.getTaxPaise();
        d.total = inv.getTotalPaise();
        d.gstPercent = inv.getGstPercent();
        return InvoicePdf.render(d);
    }

    public String fileName(Long id) {
        return "invoice-" + load(id).getInvoiceNumber() + ".pdf";
    }

    // ----- helpers

    private Invoice load(Long id) {
        Invoice inv = DatabaseOperationException.guard("loading an invoice", () -> invoices.findById(id))
            .orElseThrow(() -> new ResourceNotFoundException("Invoice not found"));
        check(inv);
        return inv;
    }

    /** An admin may see every invoice; a customer only his own. A stranger gets 404, so ids cannot be
    guessed. */
    private void check(Invoice inv) {
        if (access.isAdmin()) return;
        Customer me;
        try {
            me = access.currentCustomer();
        } catch (RuntimeException e) {
            throw new AccessDeniedException("Not allowed");
        }
        if (me == null || !me.getCustomerId().equals(inv.getCustomerId())) {
            throw new ResourceNotFoundException("Invoice not found");
        }
    }

    static List<InvoiceLineView> lines(Invoice inv) {
        List<InvoiceLineView> out = new ArrayList<>();
        if (inv.getLines() == null || inv.getLines().isBlank()) return out;
        for (String row : inv.getLines().split("\n")) {

```

```

        String[] p = row.split("\\|", 3);
        if (p.length < 3) continue;
        InvoiceLineView l = new InvoiceLineView();
        l.courseId = Long.valueOf(p[0]);
        l.title = p[1];
        l.paise = Long.parseLong(p[2]);
        out.add(l);
    }
    return out;
}

private InvoiceView view(Invoice i) {
    InvoiceView v = new InvoiceView();
    v.id = i.getId();
    v.invoiceNumber = i.getInvoiceNumber();
    v.paymentId = i.getPaymentId();
    v.orderId = i.getOrderId();
    v.razorpayPaymentId = i.getRazorpayPaymentId();
    v.customerName = i.getCustomerName();
    v.customerEmail = i.getCustomerEmail();
    v.issuedAt = i.getIssuedAt() == null ? null : i.getIssuedAt().toString();
    v.lines = lines(i);
    v.taxablePaise = i.getTaxablePaise();
    v.taxPaise = i.getTaxPaise();
    v.totalPaise = i.getTotalPaise();
    v.gstPercent = i.getGstPercent();
    v.sellerName = sellerName;
    v.sellerAddress = sellerAddress.replace("\\n", "\n");
    v.sellerGstin = sellerGstin;
    return v;
}
}

```

springapp/src/main/java/com/examly/springapp/controller/InvoiceController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.PayDtos.InvoiceView;
import com.examly.springapp.service.InvoiceService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.util.List;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/** Invoices of paid orders. A customer sees only his own; an admin sees all. */
@RestController
@RequestMapping("/api/invoice")
@Tag(name = "Invoices", description = "Invoices for verified payments")
public class InvoiceController {
    private final InvoiceService service;

    public InvoiceController(InvoiceService service) {
        this.service = service;
    }

    @Operation(summary = "My invoices, newest first")
    @GetMapping("/my")
    public ResponseEntity<List<InvoiceView>> mine() {
        return ResponseEntity.ok(service.listMine());
    }

    @Operation(summary = "The invoice of one of my orders (made on the spot if it is missing)")
    @GetMapping("/by-order/{orderId}")
    public ResponseEntity<InvoiceView> byOrder(@PathVariable Long orderId) {
        return ResponseEntity.ok(service.getByOrder(orderId));
    }
}

```

```

    }

    @Operation(summary = "One invoice as data")
    @GetMapping("/{id}")
    public ResponseEntity<InvoiceView> one(@PathVariable Long id) {
        return ResponseEntity.ok(service.get(id));
    }

    @Operation(summary = "One invoice as a PDF file")
    @GetMapping("/{id}/pdf")
    public ResponseEntity<byte[]> pdf(@PathVariable Long id) {
        byte[] bytes = service.pdf(id);
        return ResponseEntity.ok()
            .header("Content-Type", "application/pdf")
            .header("Content-Disposition", "attachment; filename=\"" + service.fileName(id) + ".pdf\"")
            .header("Cache-Control", "no-store")
            .body(bytes);
    }
}

```

springapp/src/main/java/com/examly/springapp/service/PaymentService.java (REPLACE the Day 2 file: the constructor also takes an `InvoiceService`, and `afterPaid` makes the invoice before it writes the earnings)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.RazorpayVerifyDTO;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.DuplicateResourceException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Cart;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.Customer;
import com.examly.springapp.model.Orders;
import com.examly.springapp.model.Payment;
import com.examly.springapp.repository.OrderRepo;
import com.examly.springapp.repository.PaymentRepo;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;

/**
 * Pay for the cart with Razorpay (test mode or live mode, decided only by the keys you configure).
 * <p>
 * Step 1, {@link #createCheckout}: the SERVER reads the customer's cart, adds up the prices from the
 * database
 * (the browser can never choose the price), creates an order at Razorpay and remembers it in the payments
 * table.
 * <br>
 * Step 2, {@link #verifyAndEnroll}: after the customer pays, the server checks Razorpay's signature. Only
 * then it saves
 * the real order through the existing OrderService, so the duplicate-purchase rule still applies, and
 * empties the cart.
 */
@Service
public class PaymentService {

```

```

private static final Logger log = LoggerFactory.getLogger(PaymentService.class);
/** Razorpay cannot charge less than 1 rupee (100 paise). */
private static final long MIN_PAISE = 100;

private final RazorpayClient razorpay;
private final CartService cartService;
private final OrderService orderService;
private final OrderRepo orderRepo;
private final PaymentRepo paymentRepo;
private final InvoiceService invoiceService;
private final EarningService earningService;

public PaymentService(RazorpayClient razorpay, CartService cartService, OrderService orderService,
                    OrderRepo orderRepo, PaymentRepo paymentRepo,
                    InvoiceService invoiceService, EarningService earningService) {
    this.razorpay = razorpay;
    this.cartService = cartService;
    this.orderService = orderService;
    this.orderRepo = orderRepo;
    this.paymentRepo = paymentRepo;
    this.invoiceService = invoiceService;
    this.earningService = earningService;
}

/** Creates the Razorpay order for what is in the customer's cart and returns what the browser needs to
open Checkout. */
public Map<String, Object> createCheckout(Customer customer) {
    log.trace("createCheckout entered");
    requireConfigured();
    Long customerId = customer.getCustomerId();
    Cart cart = cartService.getCartByCustomerId(customerId);
    List<Course> courses = cart.getCourses();
    if (courses == null || courses.isEmpty()) {
        throw new InvalidRequestException("Your cart is empty");
    }
    // same rule as the order service: a course can be bought only once
    Set<Long> bought = new HashSet<>();
    DatabaseOperationException.guard("loading orders", () ->
orderRepo.findByCustomer_CustomerId(customerId))
        .forEach(o -> o.getCourses().forEach(c -> bought.add(c.getCourseId())));
    BigDecimal total = BigDecimal.ZERO;
    for (Course c : courses) {
        if (bought.contains(c.getCourseId())) {
            throw new DuplicateResourceException("You are already enrolled in " + c.getCourseType());
        }
        total = total.add(BigDecimal.valueOf(c.getCoursePrice() == null ? 0 : c.getCoursePrice()));
    }
    long paise = total.movePointRight(2).setScale(0, RoundingMode.HALF_UP).longValueExact();
    if (paise < MIN_PAISE) {
        throw new InvalidRequestException("Razorpay needs a total of at least Rs 1. A free cart does
not need a payment.");
    }

    String receipt = ("rcpt_" + customerId + "_" + System.currentTimeMillis());
    String razorpayOrderId = razorpay.createOrder(paise, receipt.length() > 40 ? receipt.substring(0,
40) : receipt);

    Payment payment = new Payment();
    payment.setRazorpayOrderId(razorpayOrderId);
    payment.setAmountPaise(paise);
    payment.setCurrency("INR");
    payment.setStatus(Payment.CREATED);
    payment.setCustomerId(customerId);
    payment.setCourseIds(courses.stream().map(c ->
String.valueOf(c.getCourseId())).collect(Collectors.joining(",")));
    payment.setCreatedAt(Instant.now());
    DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));

    Map<String, Object> prefill = new LinkedHashMap<>();
    prefill.put("name", customer.getCustomerName());
    if (customer.getUser() != null) {

```

```

        prefill.put("email", customer.getUser().getEmail());
        String mobile = customer.getUser().getMobileNumber();
        if (mobile != null && mobile.matches("^[6-9][0-9]{9}$")) {
            prefill.put("contact", mobile);
        }
    }
    Map<String, Object> out = new LinkedHashMap<>();
    out.put("keyId", razorpay.getKeyId());
    out.put("razorpayOrderId", razorpayOrderId);
    out.put("amount", paise);
    out.put("currency", "INR");
    out.put("name", "Online E-Learning Platform");
    out.put("description", courses.size() + (courses.size() == 1 ? " course" : " courses"));
    out.put("prefill", prefill);
    log.info("Checkout prepared: customer {}, {} course(s), {} paise", customerId, courses.size(),
paise);
    return out;
}

/** Checks the signature, then saves the real order. Calling it twice with the same payment returns the
same order. */
public Orders verifyAndEnroll(Customer customer, RazorpayVerifyDTO dto) {
    log.trace("verifyAndEnroll entered");
    requireConfigured();
    Payment payment = DatabaseOperationException.guard("loading a payment", () ->
paymentRepo.findByRazorpayOrderId(dto.razorpayOrderId()))
        .orElseThrow(() -> new ResourceNotFoundException("Payment not found"));
    if (!payment.getCustomerId().equals(customer.getCustomerId())) {
        log.warn("Payment {} belongs to another customer", payment.getPaymentId());
        throw new AccessDeniedException("This payment belongs to another customer");
    }
    if (Payment.PAID.equals(payment.getStatus()) && payment.getOrderId() != null) {
        // the browser repeated the call (double click, reload): answer with the order we already saved
        log.info("Payment {} was already verified; returning order {}", payment.getPaymentId(),
payment.getOrderId());
        afterPaid(payment, payment.getOrderId()); // heals an invoice / earning that failed the
first time
        return orderService.getOrderByOrderId(payment.getOrderId());
    }
    if (!RazorpaySignature.matches(dto.razorpayOrderId(), dto.razorpayPaymentId(),
dto.razorpaySignature(), razorpay.getKeySecret())) {
        log.warn("Payment {}: signature check FAILED", payment.getPaymentId());
        throw new InvalidRequestException("Payment verification failed. You were not enrolled.");
    }

    // the signature is valid: the money is real. Record it before anything else can go wrong.
    payment.setRazorpayPaymentId(dto.razorpayPaymentId());
    payment.setStatus(Payment.PAID);
    payment.setPaidAt(Instant.now());
    DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));

    Orders order = new Orders();
    order.setCustomer(customer);
    List<Course> stubs = new ArrayList<>();
    for (String id : payment.getCourseIds().split(",")) {
        Course c = new Course();
        c.setCourseId(Long.valueOf(id.trim()));
        stubs.add(c);
    }
    order.setCourses(stubs);
    Orders saved;
    try {
        saved = orderService.addOrder(order); // recalculates the price from the database and applies
the duplicate rule
    } catch (RuntimeException e) {
        payment.setStatus(Payment.PAID_ORDER_FAILED);
        paymentRepo.save(payment);
        log.error("Payment {} (Razorpay payment id {}) was paid but the enrollment failed: {}",
            payment.getPaymentId(), dto.razorpayPaymentId(), e.getMessage());
        throw e;
    }
}

```

```

        payment.setOrderId(saved.getOrderId());
        DatabaseOperationException.guard("saving a payment", () -> paymentRepo.save(payment));

        try {
            cartService.removeAllCourses(cartService.getCartByCustomerId(customer.getCustomerId()).getCartId());
        } catch (RuntimeException e) {
            log.warn("Order {} saved but the cart could not be emptied: {}", saved.getOrderId(),
                e.getMessage());
        }
        afterPaid(payment, saved.getOrderId());
        log.info("Payment {} verified: order {} created for customer {}", payment.getPaymentId(),
            saved.getOrderId(), customer.getCustomerId());
        return saved;
    }

    /**
     * Invoice and instructor earnings come AFTER the money and the enrollment are safe. If either fails,
     * the
     * customer is still enrolled; the error is logged and the next call (or opening the invoice) makes it
     * again.
     */
    private void afterPaid(Payment payment, Long orderId) {
        try {
            invoiceService.ensureForPayment(payment);
        } catch (RuntimeException e) {
            log.error("Payment {}: the invoice could not be made: {}", payment.getPaymentId(),
                e.getMessage());
        }
        try {
            earningService.record(payment, orderId);
        } catch (RuntimeException e) {
            log.error("Payment {}: the instructor earnings could not be recorded: {}",
                payment.getPaymentId(), e.getMessage());
        }
    }

    private void requireConfigured() {
        if (!razorpay.isConfigured()) {
            throw new InvalidRequestException("Razorpay is not set up on this server");
        }
    }
}

```

4. Check it

4a. Start the backend. No red error; Hibernate creates `invoices`.

4b. One sale, then the invoice. 1. Pay for a course with a test card (Day 1). 2. Database:

```

SELECT id, invoice_number, payment_id, order_id, customer_name, total_paise, tax_paise, taxable_paise,
gst_percent, lines
FROM invoices ORDER BY id DESC;

```

You should see `INV-2026-000001`, the total in paise, and the lines text. 3. In Swagger (log in as that customer, click Authorize): `GET /api/invoice/my` lists it. `GET /api/invoice/1/pdf`: Swagger shows a **Download file** link; the file opens in any PDF viewer. 4. With `INVOICE_GST_PERCENT=18` and a seller GSTIN, the PDF title becomes **TAX INVOICE** and shows "Amount before tax" and "GST 18% (included in price)".

4c. Things to try (these are what the tests checked).

Try	Expected
Verify the same payment twice	still one invoice
Rename a course after paying, then open the invoice	the old course name is still on it
Open another customer's invoice id	404 "Invoice not found"
Open it as admin	works
Open it as an instructor	403 (Security rule)
GET <code>/api/invoice/by-order/{orderId}</code> for an order with no payment	404 "There is no invoice for this order"
Make the database refuse the invoice write (test only)	the student is still enrolled; asking for the invoice again makes it
A cart of 30 courses	the PDF has 2 pages

5. If something goes wrong

What you see	Why	Fix
Start-up error "No qualifying bean of type InvoiceService" or <code>PaymentService</code> constructor	<code>PaymentService</code> was not replaced by the Day 3 version	replace it
Unable to find a method ... <code>findByOrderId</code>	<code>PaymentRepo</code> was not replaced	replace it
PDF opens blank or "file is damaged"	an edit in <code>InvoicePdf.java</code> broke the byte counting	paste the file again, do not edit the numbers
PDF shows <code>?</code> in a name	the name has a character outside plain English letters	expected with the built-in font; see section 6
Invoice shows no GST lines	<code>invoice.gst-percent</code> is 0	set <code>INVOICE_GST_PERCENT=18</code> (only if your company charges GST)
404 on <code>/by-order/..</code> right after paying	the order was not paid through Razorpay	pay with Razorpay (or set <code>razorpay.enforce=true</code>)
Invoice number jumps (1, 2, 4)	a failed save used an id	harmless; ids are unique, not gap-free

6. What is not built (honest list)

- **Credit notes / refunds:** no invoice is cancelled or reversed.
- **Non-Latin characters** (for example Telugu or Hindi course names) print as `?` in the PDF, because we use the built-in font. Fixing it means embedding a font file (a library such as OpenPDF/iText plus a `.ttf` file). The on-screen invoice shows every language correctly.
- **The rupee sign** is written `Rs` in the PDF for the same reason.
- **E-mailing the invoice** to the customer: not built.
- **GST rules** (HSN/SAC code, place of supply, CGST+SGST vs IGST split): not built; the invoice shows a single GST line.
- **Seller details** are read from `application.properties` when the PDF is drawn, so changing them changes how **old** invoices print their header. The buyer, lines and amounts never change.

Honest status. Invoice creation, numbering, tax split, access rules, the PDF writer and the whole pay-and-verify flow ran as real Java against in-memory stand-ins: **136 checks, 0 failures** (shared with Day 2). The PDF output was opened with a real PDF reader (`pdfinfo` , `pdftotext`) and rendered to an image: 1 page for 2 courses, 2 pages for 30. **Not tested:** Spring wiring against a real MySQL, the real Razorpay payment.